15,471,856 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

Messaging with RabbitMQ

Keyhole Software
13 May 2013 CPOL

Rate me: 4.14/5 (5 votes)

This article introduces the relevant concepts, hopefully helps you to navigate around one or two common “gotchas,” and provides links to more in-depth resources.

We previously published a brief tutorial showing how to set up a Spring RabbitMQ Template. Now, let’s take a step back for a broader view of what RabbitMQ is and how you might use it.

This article introduces the relevant concepts, hopefully helps you to navigate around one or two common “gotchas,” and provides links to more in-depth resources. It doesn’t get into the “nuts and bolts” of each usage pattern or of coding with any particular client library; see the tutorials linked at the end of the article for information on those topics.

What Is RabbitMQ

RabbitMQ is messaging software built on the AMQP protocol. Applications and services can exchange messages via queues managed by a RabbitMQ **broker**.

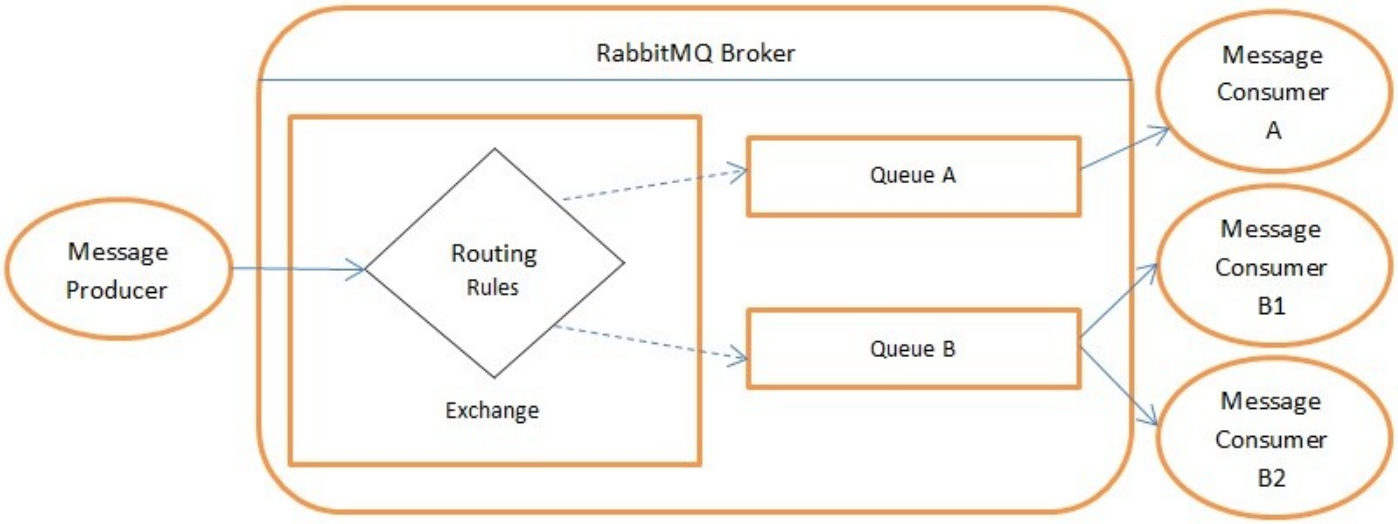
There are many potential use cases for RabbitMQ. It can be used to dispatch tasks to be handled by a specialized service synchronously (as an RPC mechanism) or asynchronously (work queues). It can be used to broadcast information about events, such as task status updates that might be monitored by logging services and dashboard applications (publish/subscribe).

Message Queues and Exchanges

To users of other message queuing systems, a few things about RabbitMQ may be surprising. Queues can be created dynamically by message producers and/or message consumers; in fact, patterns like

publish/subscribe rely on each consumer to create its own queue from which to read messages, which at first may seem counterintuitive.

Message producers don't interact directly with queues; instead they send messages to **exchanges**. Each exchange may be bound to any number of queues. There are several types of exchange, each with its own set of routing rules that determine which of its bound queues, if any, will receive a given message.



Any number of consumers may listen for messages on a queue (unless the queue was set up by a consumer for its own exclusive use). This is **not** a publish/subscribe mechanism, however, because any given message placed on the queue will be delivered to only one of the consumers listening on that queue. By default a round-robin approach is used to decide which consumer gets each message; you can configure the queue such that a given consumer is ineligible to receive more messages until it is done processing any messages it's already received. This can be useful for work queues to ensure a "fair" distribution of work.

Messages

Per the AMQP protocol, a message consists of a set of properties (basically key-value pairs) and a payload. (The protocol actually refers to this as the **bare message**; additional packaging is added as the message moves through the RabbitMQ infrastructure.) The payload can contain basically anything. One flexible approach is to use JSON-encoded maps containing whatever data your applications/services need to share; the Spring packages provide good support for this approach, or for using JSON-encoded objects in general. This is not required, though; whatever your consumers and producers agree on is acceptable.

There are a handful of standard properties, and applications can define their own properties (sometimes called "**headers**") as well. Headers can mean whatever the producer and consumer want them to mean. (This may seem redundant; any application-defined information can be embedded in the payload, so why use application-defined properties? Headers can make sense, though, for information that is logically metadata describing the message.) The conventional wisdom seems to be

that even most of the standard properties are open to interpretation by the application, but a certain amount of caution is advisable as some of them have special meaning to the broker.

The best advice is to read up on the standard properties in the AMQP specification so that you'll recognize when one of them might serve your intended purpose. Referring to a protocol specification may seem daunting, but this one is reasonably accessible, and other sites – even those that otherwise contain pretty good tutorials – tend to gloss over the standard message properties. Misuse of a standard property, if the broker happens to use that particular value, can cause strange and unexpected behaviors.

You're rarely required to directly set the standard properties, as a good client library will manage them adequately. When in doubt, the safest course is to use application-defined properties or the message payload for the information your message consumers will need.

Administration

RabbitMQ provides a web-based administration interface. Through this interface you can manager users and their permissions; monitor connections, exchanges, queues, and so on; send test messages or examine messages that are waiting in queues; etc. Debugging the interactions of message producers and consumers becomes much easier if you familiarize yourself with this interface.

While the administration interface is great for monitoring the overall behavior of the broker over time, it doesn't generally provide good visibility of instantaneous events like errors that might occur when accepting a message. You should also be familiar with RabbitMQ's log output.

Users have three types of permission – read, write, and configure – each consisting of a regular expression. Each of these permissions governs certain corresponding operations; for example, the configure operations governs the ability to create and delete named resources (queues and exchanges). A user is permitted to perform an operation on a given resource if the resource name matches the pattern set for the corresponding permission.

In the most permissive use case, a user's read, write, and configure permissions can all be set to "**.***". This means that the user can perform any operation (create, delete, read to, write from, etc.) on any resource. Fully dynamic queue creation is possible; a client simply declares that it wants to use a queue or exchange, and if no queue or exchange exists with that name then one is created.

At the other extreme, you could set the configure permission to "**^$**", which won't match any valid resource name, for all users except specially designated queue administrators. This would simulate a more traditional approach where all the queues you're going to use are created in advance. This may suit some purposes, but will not work well with patterns like publish/subscribe the way they're typically done in RabbitMQ.

A middle-ground approach might give each user a prefix to be used in naming resources, to avoid naming collisions amongst applications. For example appA may log in as app1User, and app1User's "configure" permission can be set to "appA_.*".

More information about RabbitMQ permissions can be found here.

An obvious concern with dynamic queue creation is the possibility that a bunch of stale queues might be left idling, especially if some unforeseen event causes a client to lose connectivity. A queue can be configured as "exclusive" – meaning that only the client that created it may read from it – in which case it will be deleted automatically when that client disconnects. Or, even a shared queue can be marked for auto-deletion when no one is listening to it any more.

Further Reading

The RabbitMQ "Getting Started" page provides tutorials explaining the typical usage patterns and how to implement them using the standard Python or Java client libraries.

More generally, the RabbitMQ web site has a lot of useful information. You may want to look over the resources linked from the Developer Tools section of the Community page, as well as the various documents on the Documentation page.

The Spring AMQP project provides an alternative way to work with RabbitMQ, which can be simpler to work with (especially in Spring-based project environments). Per the Spring philosophy, it can take care of (or at least simplify) many of the rote tasks involved in using RabbitMQ.

– Mark Adelsberger, asktheteam@keyholesoftware.com.

This article was originally posted at http://keyholesoftware.com/2013/05/13/messaging-with-rabbitmq

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Keyhole Software

Keyhole Software

United States

Keyhole is a software development and consulting firm with a tight-knit technical team. We work primarily with Java, .NET, and Mobile technologies, specializing in application development. We love the challenge that comes in consulting and blog often regarding some of the technical situations and technologies we face. Kansas City, St. Louis and Chicago.

This is a Organisation

3 members

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Thanks

Alireza_1362 23-Apr-16 15:02

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2013 by Keyhole Software
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1